
Smart Campus Service Platform

Project Report

Daniel Simbe

July 2026

1. Introdução

A Smart Campus Service Platform é uma plataforma distribuída de monitorização e colaboração entre serviços de um campus universitário. O projeto simula um conjunto de serviços, como salas, estacionamento, impressão, biblioteca, transportes e cantina, permitindo observar o seu estado e a comunicação entre os diferentes componentes do sistema.

A comunicação é baseada em eventos e utiliza MQTT como mecanismo de troca de mensagens. A plataforma acompanha o ciclo de vida dos serviços, mede a latência através de RTT e disponibiliza informação operacional através de dashboards. A separação entre estado em tempo real e histórico permite também analisar o comportamento da plataforma ao longo do tempo.

2. Objetivos

O objetivo principal é desenvolver uma plataforma de monitorização orientada a eventos, capaz de acompanhar serviços distribuídos e apresentar o seu estado operacional de forma clara.

Como objetivos específicos, pretende-se:

- Simular serviços distribuídos de um campus universitário.
- Implementar descoberta dinâmica e acompanhamento do ciclo de vida dos serviços.
- Medir disponibilidade e latência através de RTT.
- Utilizar MQTT para comunicação assíncrona baseada em Publish/Subscribe.
- Manter estado operacional em tempo real e histórico persistente.
- Disponibilizar observabilidade através de interfaces web e terminal.
- Demonstrar colaboração entre serviços através de eventos de domínio.

3. Arquitetura da Plataforma

A arquitetura é composta por agentes, um broker MQTT, um monitor central, dois mecanismos de armazenamento e interfaces de observabilidade. A comunicação entre os principais componentes segue um modelo distribuído e orientado a eventos.

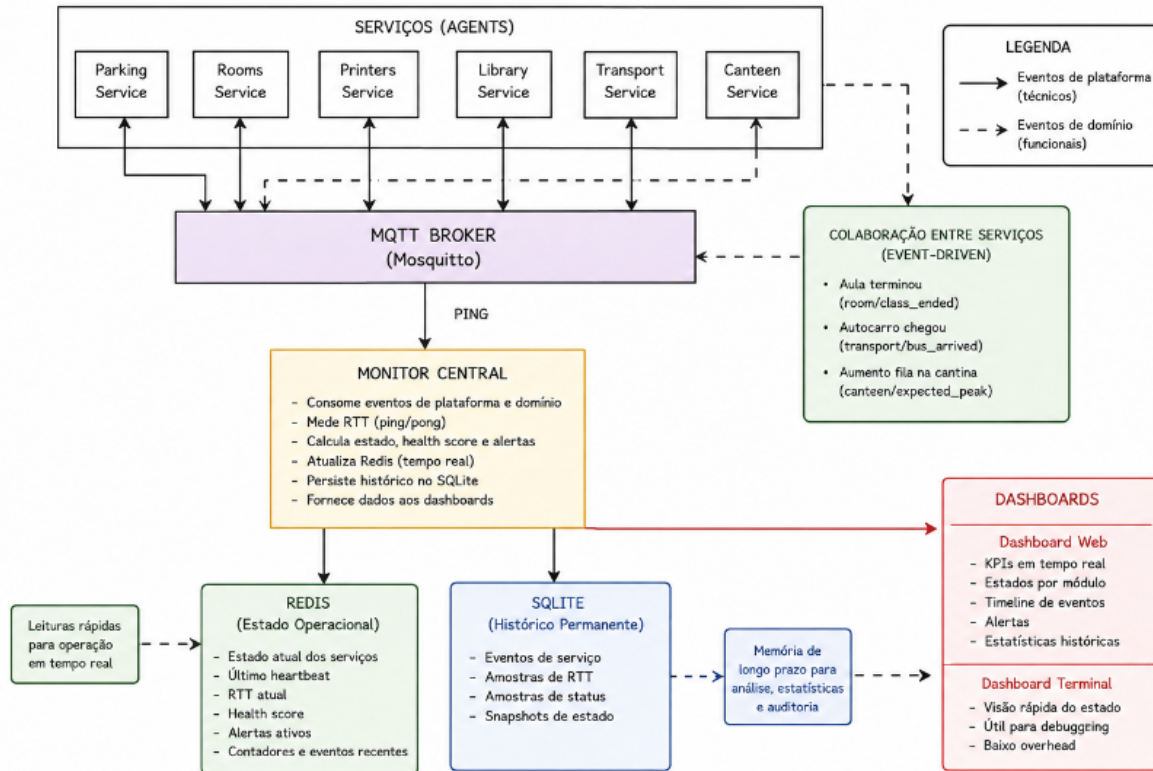


Figura 1 — Arquitetura simplificada da plataforma.

Os agentes representam os serviços do campus e comunicam através do MQTT. O broker Mosquitto funciona como intermediário das mensagens, evitando que os serviços tenham de estabelecer ligações diretas entre si.

O monitor central é responsável por observar a plataforma, processar os eventos recebidos, medir o RTT e determinar o estado operacional dos serviços. O Redis mantém o estado atual do sistema, enquanto o SQLite conserva informação histórica.

A informação é disponibilizada através de um dashboard web e de uma interface de terminal. Esta separação permite distinguir a lógica de comunicação e monitorização da sua visualização.

4. Comunicação e Funcionamento

A comunicação da plataforma é baseada no modelo Publish/Subscribe do MQTT. Os agentes publicam mensagens nos tópicos correspondentes aos seus serviços e os componentes interessados subscrevem esses tópicos. O broker Mosquitto é responsável por distribuir as mensagens aos subscritores.

4.1 Ciclo de vida dos serviços

Cada agente possui um ciclo de vida que permite ao monitor acompanhar a sua presença na plataforma. No arranque, o agente publica REGISTER e METADATA. Durante a execução, envia periodicamente HEARTBEAT e STATUS. O heartbeat indica que o serviço continua ativo, enquanto o status transporta informação funcional específica do módulo.

Para medir a latência, o monitor envia mensagens PING e o agente responde com PONG. No encerramento controlado, o agente publica UNREGISTER.

REGISTER → METADATA → HEARTBEAT / STATUS → PING / PONG → UNREGISTER

4.2 Eventos de plataforma e eventos de domínio

A plataforma distingue eventos de plataforma de eventos de domínio. Os primeiros estão relacionados com o funcionamento técnico dos serviços, como o registo, o heartbeat ou a resposta a um ping. Os eventos de domínio representam acontecimentos funcionais do campus, como o fim de uma aula ou a chegada de um transporte.

Através dos eventos de domínio, os serviços podem reagir a acontecimentos relevantes sem comunicação direta entre si. Assim, a colaboração é realizada através do broker MQTT e mantém o desacoplamento entre os agentes.

4.3 Disponibilidade e RTT

O monitor considera um serviço ativo quando recebe informação válida do agente. A ausência de heartbeats durante o período de timeout configurado permite detetar uma falha e marcar o serviço como DOWN. Um encerramento controlado é identificado através do evento UNREGISTER.

A latência é medida através do RTT, calculado como o tempo entre o envio do PING e a receção do PONG. As medições são utilizadas na observabilidade do sistema e são também armazenadas para análise histórica.

5. Simulação dos Serviços

Como a plataforma não está ligada a sensores ou sistemas reais do campus, os agentes simulam o comportamento dos serviços. A simulação foi criada para gerar informação dinâmica e permitir testar os mecanismos de comunicação e monitorização.

Os valores funcionais variam ao longo do tempo de acordo com perfis de utilização definidos para cada módulo. São introduzidas pequenas variações controladas para evitar que os valores permaneçam constantes ou sofram alterações completamente aleatórias.

Serviço	Exemplos de dados
Rooms	Salas livres e ocupadas
Parking	Lugares livres e ocupados
Printers	Trabalhos e tamanho da fila
Canteen	Fila média e refeições disponíveis
Transport	Paragem atual e ETA
Library	Ocupação e lugares livres

Tabela 1 — Exemplos de informação funcional simulada.

A simulação permite representar alterações na procura de um serviço e observar o seu comportamento através dos eventos gerados. Desta forma, os agentes não funcionam apenas como processos que enviam mensagens técnicas, mas como serviços com comportamento funcional simulado.

6. Observabilidade

O monitor central constitui o núcleo de observabilidade da plataforma. A partir dos eventos recebidos através do MQTT, acompanha o estado dos serviços, mede a latência e agrega informação sobre o funcionamento do sistema.

A informação processada é disponibilizada através de duas interfaces. O dashboard web apresenta uma visão global da plataforma, com estados por módulo, métricas, alertas e eventos recentes. A interface de terminal fornece uma visão mais simples e técnica, útil para testes e acompanhamento rápido do sistema.

A plataforma também distingue diferentes estados de saúde dos serviços, permitindo identificar situações normais ou anómalas. A observabilidade não se limita à indicação UP/DOWN, uma vez que também considera informação de latência e estado operacional.

7. Estado e Persistência

A plataforma separa o estado operacional atual da informação histórica. Esta decisão permite responder de forma diferente a consultas em tempo real e a consultas de análise.

Tecnologia	Função
Redis	Estado operacional atual e consultas rápidas
SQLite	Persistência de eventos e métricas históricas

Tabela 2 — Função dos mecanismos de armazenamento.

O Redis é atualizado pelo monitor com a informação mais recente dos serviços. Guarda, entre outros dados, o estado atual, o último heartbeat, o RTT e os alertas ativos. É utilizado quando a plataforma necessita de uma visão rápida do estado atual.

O SQLite conserva eventos, amostras de RTT e informação de estado. A persistência permite calcular estatísticas e analisar o comportamento da plataforma ao longo do tempo.

8. Tolerância a Falhas

Foram considerados diferentes cenários de falha para avaliar o comportamento da arquitetura. Os clientes MQTT possuem mecanismos de reconexão e os containers dos serviços críticos podem ser reiniciados automaticamente.

A falha de um agente é isolada dos restantes serviços. O monitor deteta a ausência de heartbeat e assinala o serviço como indisponível. Quando o agente recupera, volta a comunicar com a plataforma e o seu estado pode ser reconstruído.

Se o monitor falhar, os serviços continuam a existir e a comunicar através do MQTT, mas a plataforma perde temporariamente a capacidade de observação e agregação. A falha do broker MQTT tem um impacto mais abrangente, uma vez que este é o ponto central da comunicação atual.

9. Testes e Resultados

A validação foi realizada através de testes manuais orientados aos principais mecanismos da plataforma. Foram testados cenários normais e situações de falha.

Cenário	Resultado observado
Queda de um agente	Monitor deteta a indisponibilidade através do timeout.
Recuperação de um agente	Serviço volta a comunicar e a ser monitorizado.
Medição de RTT	Monitor calcula e apresenta a latência.
Eventos de domínio	Eventos são recebidos e observados no dashboard.
Queda do monitor	Observabilidade é interrompida até à recuperação.
Queda do broker	Comunicação MQTT deixa de funcionar.

Tabela 3 — Resumo dos testes manuais realizados.

Os testes confirmaram o funcionamento dos principais mecanismos de comunicação e monitorização. A plataforma conseguiu detetar alterações no estado dos serviços, calcular latência e apresentar os eventos recebidos nas interfaces de observabilidade.

10. Limitações e Melhorias Futuras

A principal limitação atual é a utilização de um único broker MQTT, que constitui um ponto único de falha na comunicação. Além disso, os dados funcionais são simulados e não provêm de sistemas reais do campus.

Como evolução futura, poderá ser implementada uma arquitetura de alta disponibilidade para a mensageria, bem como a integração com fontes de dados reais. Também poderão ser acrescentados testes automatizados de integração e regras de alerta mais específicas para cada serviço.

11. Conclusão

A Smart Campus Service Platform implementa uma plataforma distribuída de monitorização baseada em comunicação assíncrona através de MQTT. A arquitetura permite representar serviços de campus, acompanhar o seu ciclo de vida, detetar falhas e medir a latência da comunicação.

O monitor central agrega a informação recebida e fornece uma visão operacional do sistema, enquanto o Redis e o SQLite permitem separar o estado atual do histórico. A existência de dashboards web e terminal facilita a observação e validação do comportamento da plataforma.

Os resultados obtidos demonstram que a solução cumpre os objetivos definidos e constitui uma base adequada para a evolução de uma plataforma distribuída de observabilidade aplicada a um cenário de Smart Campus.